# PS4a: Static Animation Player

Inbetweening (usually shortened to just tweening) is an animation process where certain known key frames are created and then intermediate frames are created by interpolating between those key frames to fill in the missing data. This allows for smooth animation without having to manually specify each individual frame.

In traditional animation, a key animator would draw out the important key frames and then a separate artist will fill in the gaps. Usually, this would be done "on twos" where the same frame would be repeated twice to reduce work with only very fast movement being drawn "on ones" with separate images every frame. For computer-generated animation, usually every frame is animated with lower frame rates being a conscious artistic choice.

In this assignment, you will load animation data from a file and draw it to the screen. You will add motion and tweening in part B.

For this assignment, you may work with a partner as long as you use pair programming techniques.

## 1 Details

Your task is to create a program AniPlayer that will take a file name as a command-line argument and play a video described by that file. For Part A of this assignment, you will load the video and draws the first frame. In Part B, you will add tweening and movement.

### 1.1 JSON

The video data is stored in JSON (JavaScript Object Notation) format. JSON is a file format that uses human-readable text to represent field-value pairs. While JSON was derived from JavaScript syntax, it is a language-independent data format. Many modern programming languages have libraries to generate and parse JSON data.

The nlohmann/json library (primarily by Niels Lohmann) is one of the more popular JSON libraries for C++. You can install it with:

```
sudo apt install nlohmann-json3-dev
```

JSON data contain objects represented by comma-separated key-value pairs encased within curly braces, similar to a map/dictionary. The ordering of the pairs has no special significance. Keys are always strings, but values can be numbers, strings, booleans, an array (using square brackets), or another object. Whitespace is ignored outside of string values.

```
{
  "shape": "circle",
  "radius": 50,
  "keyframes": [
    {"t":0, "x":50, "y":50},
    {"t":5, "x":300, "y": 300}
  ]
}
```

This example is an object with three fields, `"shape"`, `"radius"`, and `"keyframes"`. Its `"shape"` property is a `string` with a value of `"circle"`, its `"radius"` property is a number with a value of 50, and its `"keyframes"` field is a list that contains two more objects.

The `json::parse` factory will produce a `json` object when given either a `istream` handle to a JSON file or a `string` that contains valid JSON. Once loaded, the `json` object has a `map`-like interface that allows for easy access of the fields, with templates providing type conversion to the proper data types. The `jsonsample.cpp` file gives an example of reading from a JSON file.

## 1.2 Implementation

For this project, you'll create a namespace `AP` that has `AniPlayer` and `KeyFrame` classes, as well as a `Component` abstract class and its concrete subclasses.

The fields of the various types in the `json` file are as follows. Note that the values in the `json` object do not necessarily have to correspond directly to variables in your class.

### 1.2.1 AniPlayer

| Field | Type | Default | Notes |
|---|---|---|---|
| width | `unsigned int` | required | |
| height | `unsigned int` | required | |
| scene | `Component` | required | |
| bgm | `string` | optional | For XC |

### 1.2.2 KeyFrame

| Field | Type | Default | Notes |
|---|---|---|---|
| t | `float` | required | |
| x | `float` | 0 | |
| y | `float` | 0 | |
| scale-x | `float` | 1 | Ratio |
| scale-y | `float` | 1 | Ratio |
| theta | `float` | 0 | In degrees |

### 1.2.3 Component

| Field | Type | Default | Notes |
|---|---|---|---|
| shape | `string` | required | |
| color | `string` (hex) | `"FFFFFFFF"` (White) | Can be 6 (rgb) or 8 (rgba) hex characters |
| keyframes | `KeyFrame[]` | Empty | List |

### 1.2.4 CircleComponent

| Field | Type | Default | Notes |
|---|---|---|---|
| radius | float | 0 | Centered at position |
| sides | `size_t` | 30 | `CircleShape` default |

### 1.2.5 RectangleComponent

| Field | Type | Default | Notes |
|---|---|---|---|
| x | `float` | 0 | |
| y | `float` | 0 | |
| width | `float` | 0 | |
| height | `float` | 0 | |

### 1.2.6 ImageComponent

| Field | Type | Default | Notes |
|---|---|---|---|
| file | `string` | required | Relative path (e.g `"images/face01.png"`) |

### 1.2.7 TextComponent

| Field | Type | Default | Notes |
|---|---|---|---|
| text | `string` | Empty | |
| font | `string` | required | Relative path |

### 1.2.8 CompositeComponent

| Field | Type | Default | Notes |
|---|---|---|---|
| children | `Component []` | Empty | List |

The `CircleComponent` and `ImageComponent` should draw with their centers at the given positions whereas the `TextComponent` should draw with its upper-left corner at the given location. The `CompositeComponent`'s children should be drawn relative to the `CompositeComponent`.

## 1.3 Factories

You will be creating multiple different kinds of `Component`s. In order to construct one, you need to know which kind needs to be constructed, which will change everytime. You could use a conditional branch to determine which kind is to be constructed, but this will result in a lot of duplicated code as every place that creates a new `Component` would need the same conditional. This makes it likely that you will forget to update one of them, causing problems.

A factory can alleviate this problem by moving the code into a single function that will construct and return the `Component`; the client will then have the factory construct the object on its behalf. At the most basic level, the factory acts as an alternative constructor that can substitute the most appropriate version of the object to be constructed instead of being locked into a particular implementation. More complicated factory objects can be configured (either through their constructors or mutators) with customizations (such as wood type or stain color) that it will use to produce several identical objects (such as chairs). In other uses, the abstract factory pattern allows for producing multiple related objects with a similar theme (such as an entire furniture set with the same overall design choices).

In this assignment, you will have a `static` factory function called `fromJson` which will take a `json` object and construct the appropriate kind of `Component`. This way clients, such as the `AniPlayer`, do not have to themselves know which kind of `Component` should be constructed. Your `fromJson` factory will throw an exception if asked to create an unknown `Component` type.

Since a factory can return different kinds of objects, they normally return pointers to the object rather than the object itself (which would be sliced). To help with memory management, `fromJson` will return a `unique_ptr`, a *smart pointer* that has sole ownership of the `Component` (as it was freshly created in the factory). Clients are free to either continue using a `unique_ptr` (if they maintain sole ownership of the `Component`) or transfer it to a `shared_ptr` (if they need to share it with other objects).

## 1.4 Composite Pattern

Your animation is going to be composed of many figures, some of which are related. For example, a single character may be composed of a torso, head, two arms, and two legs. It would be convenient if these parts could be treated as a single entity and moved in unison, with each appendage then defined by how it is different from the body rather than having to define their position in absolute coordinates.

The composite pattern allows for composing several entities together and treating them as a single object rather than having to work with them individually. At its core, there will be a base `Component` interface that declares the operations that are in common (such as the `draw`

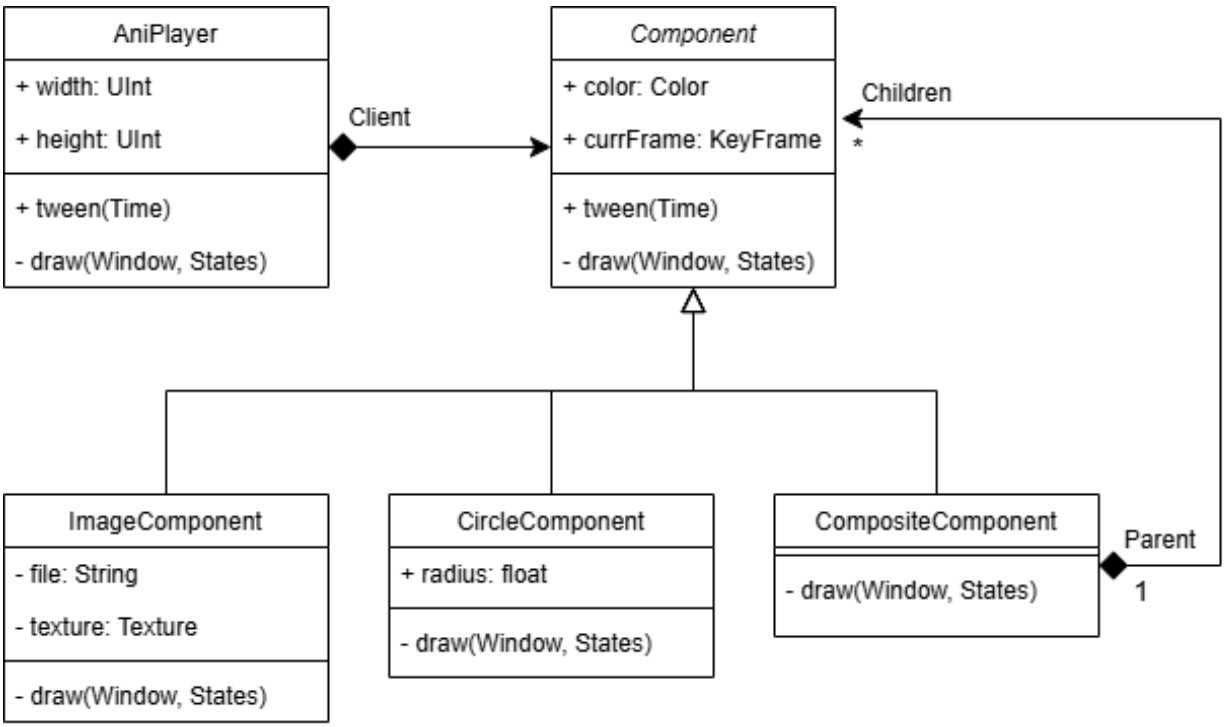



Figure 1: A `CompositeComponent` is made up of several `Component`s.

method in our case) and may provide default behavior for those operations if appropriate. Its subclasses will fall into two categories, leaves (such as your `ImageComponent`) which handle a single item, and composites (such as your `CompositeComponent`) which define behavior for storing child components. The client will store a `Component` pointer and will interact with it the same way no matter what kind of `Component` it actually points to. A sample partial UML diagram for this can be seen in 1.

## 1.5 Null Object Pattern

Whenever you have pointers, there is always the possibility if having a `nullptr` and checking for this can cause complications (using smart pointers does not change this). For example, you might have to check if a child pointer is a `nullptr` before drawing it or calling some other method on it. In these cases, it might be beneficial to have a special null object that represents the idea of not having an item. Unlike the `nullptr`, the null object is an actual object that implements the same interface as other more interesting objects. For example, it might have an empty `draw` method that does nothing and getters that return basic values (like having a `size` of 0). This results in simpler client code since you do not usually need to check for `null` values; the null object quietly does nothing instead of having to explicitly check before doing nothing.

When you get started, it may be helpful to have a `NullComponent` that does this empty behavior. Your `fromJson` factory can return a `NullComponent` in place of any kind that you haven't yet implemented. While your `fromJson` factory will throw an exception if asked to

create an unknown kind of `Component`, you may return a `NullComponent` instead of throwing for recognized but not yet implemented components so that you can render the other portions of mixed-type videos.

# 2 Development Process

There are a lot of parts to this assignment. We'd suggest the following incremental development process:

1. Stub out the `AniPlayer`, `KeyFrame`, and `Component` classes (but not any of the subclasses). Put dummy return values in their methods, but otherwise leave them empty. You can add the `public` fields to `KeyFrame`, but don't add fields to the other classes yet.
2. Setup your `main` function and have it draw the (empty) `AniPlayer` to the window.
3. Create the `NullComponent` class. It will provide empty implementations of the `Component` methods.
4. Stub out the `fromJson` factory static function. Have it return a `NullComponent`.
5. Pick one of the concrete `Component` subclasses (`CircleComponent` is probably easiest). Have its constructor read the fields specific to that `Component` from the `json`, but not any of the ones that apply to all `Component`s.
6. Edit the `fromJson` function so that it can return one of your new `Component`s.
7. Update the `AniPlayer` class so it can have and draw a `Component`. Verify that your `Component` draws itself correctly (but in the wrong place since you don't have keyframes yet).
8. Update the `KeyFrame` class so it can read its fields from the `json`.
9. Update the `Component` base class so that it can read its key frames from the `json`. Have the `currFrame` method return the first one (you will change that in Part B).
10. Implement the `KeyFrame::transform` method. Have your `Component` use that to update its states. Verify that it draws itself in the correct location.
11. Create the `CompositeComponent` class. It will need a collection of `Component` pointers.
12. Implement the `CompositeComponent::draw` method to call the `draw` method of its children. Verify that it looks correct.
13. Have the `Component` read its other fields. Make sure that your components now draw as you expect.
14. Add the other types of `Component`s. Make sure that each of them draws correctly.
15. Update the `fromJson` function to throw an exception instead of returning a `NullComponent` if the type isn't recognized.

16. Have the `AniPlayer` read its other fields. Make sure that the window is sized correctly.

# 3 Acknowledgments

**You must list any images or other resources submitted in the acknowledgments section of your readme**. Failure to do so will result in a significant penalty. For convenience, the provided resources have already been listed for you.

Note that Gradescope will remove the `videos` and `images` directories from your submission (and provide fresh copies for the autograder). This will reduce the size of your submission. However, any resources you supply for your video must be in a different directory or Gradescope will prune them.

# 4 Extra Credit

For extra credit, read the `"bgm"` field from the file and play the background music in the given file. This field should be optional (not having a `"bgm"` field should result in no music playing).

Additionally, you can allow users to specify colors by name instead of by hex code. You should recognize at least the 16 basic colors from the W3C specification, but you may optionally add additional color names (such as from the HTML 4 extended color pallet). You must still continue to support hex codes.

If you do any of the extra credit work, make sure to describe exactly what you did in `Readme-ps4.md`.

# 5 What to turn in

Your makefile should build a program named `AniPlayer` and a static library `AniPlayer.a`.

Submit a zip archive to Blackboard containing:

- Your main file `main.cpp`
- Your AniPlayer (`AniPlayer.cpp`, `AniPlayer.hpp`) class
- Your KeyFrame (`KeyFrame.cpp`, `KeyFrame.hpp`) class
- Your Component (`Component.cpp`, `Component.hpp`) class
- Any other files for the child component classes
- The makefile for your project. The makefile should have targets `all`, `AniPlayer`, `AniPlayer.a`, `lint` and `clean`. Make sure that all dependencies are correct.
- Your `Readme-ps4.md` that includes
    1. Your name

2. Statement of functionality of your program (e.g. fully works, partial functionality, extra credit)
3. Key features or algorithms used
4. Any other notes

- Any other source files that you created.
- Any images or other resources not provided (such as for the extra credit)
- A screenshot of program output

Make sure that all of your files are in a directory named `ps4a` before archiving it and that there are no `.o` or other compiled files in it.

# 6 Grading rubric

| Feature | Points | Comment |
|---|---|---|
| Autograder | 33 | |
| | 2 | Contains all code files and builds |
| | 2 | `CircleComponent` |
| | 2 | `RectangleComponent` |
| | 2 | `TextComponent` |
| | 2 | `ImageComponent` |
| | 4 | Position of first key frame |
| | 4 | `CompositeComponent` + each other component |
| | 4 | Children are placed relative to parent composites |
| | 3 | Reads and applies the `"color"` field |
| | 2 | Scaling |
| | 2 | Rotation |
| | 2 | Handles default values |
| | 2 | Detects invalid component shapes |
| Drawing | 10 | |
| | 2 | Loads filename given as program argument |
| | 3 | Window sized to given dimensions |
| | 5 | Draws first frame to window correctly |
| Documentation | 7 | |
| | 2 | Describes the algorithm or data structures used. |
| | 2 | Describes how child components were drawn. |
| | 2 | Descrbies how other features were implemented. |
| | 1 | Screenshot representative of behavior. |
| Extra Credit | 4 | |
| | +2 | Plays requested background music |
| | +2 | Supports W3C basic colors (+1 for SFML color constants only) |
| Penalties | | |
| | -25 | Uses unsourced images or other resources |
| | -5 | Memory leaks or issues |
| | -5 | Linting problems |
| | -3 | Non-private fields |
| | -10% | Each day late |
| Total | 50 | |

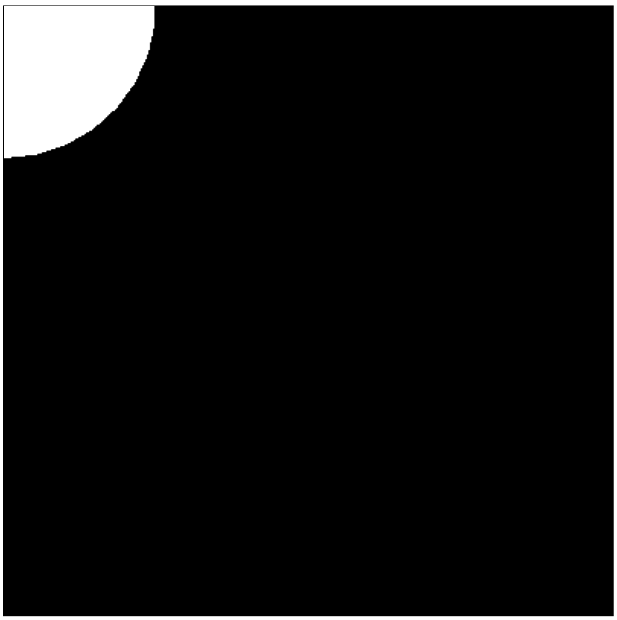


Figure 2: corner-circle-big

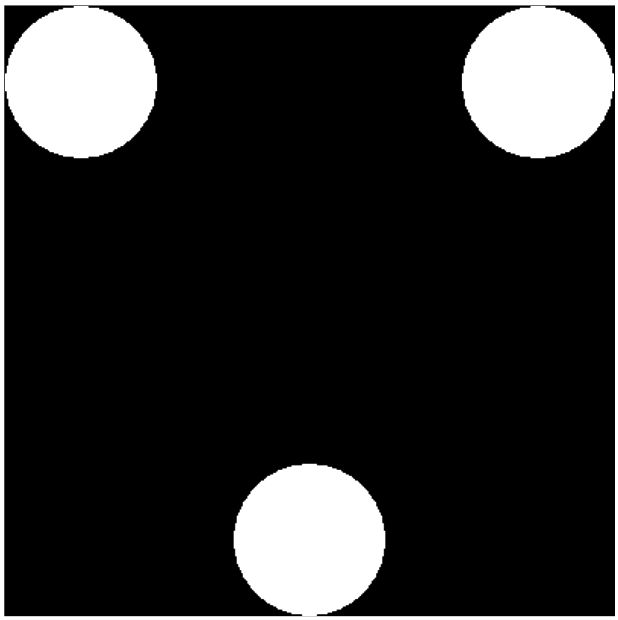


Figure 4: three-circle-a

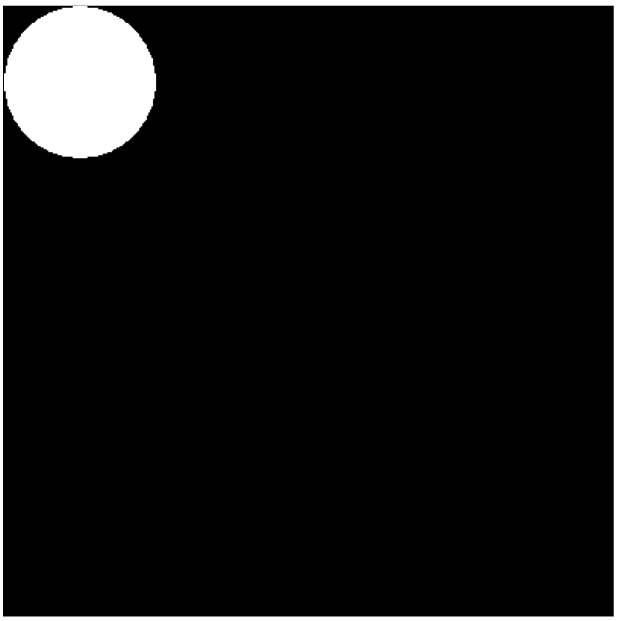


Figure 3: circle

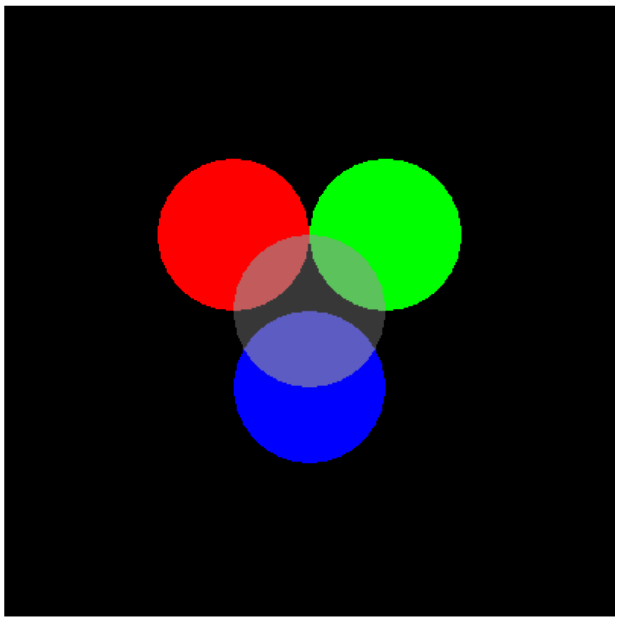


Figure 5: circle-color

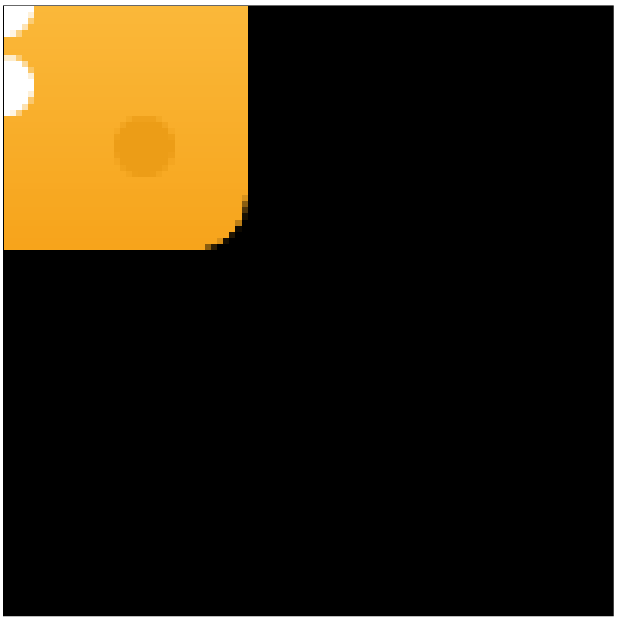


Figure 6: corner-exclamation

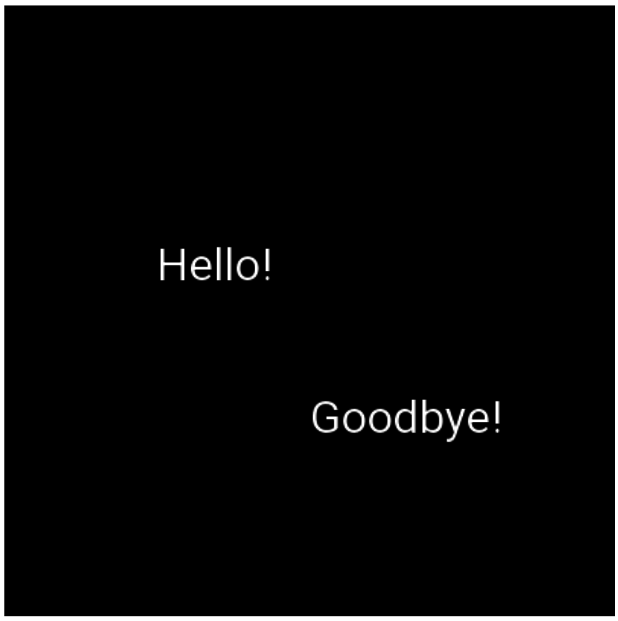



Figure 8: hello-world

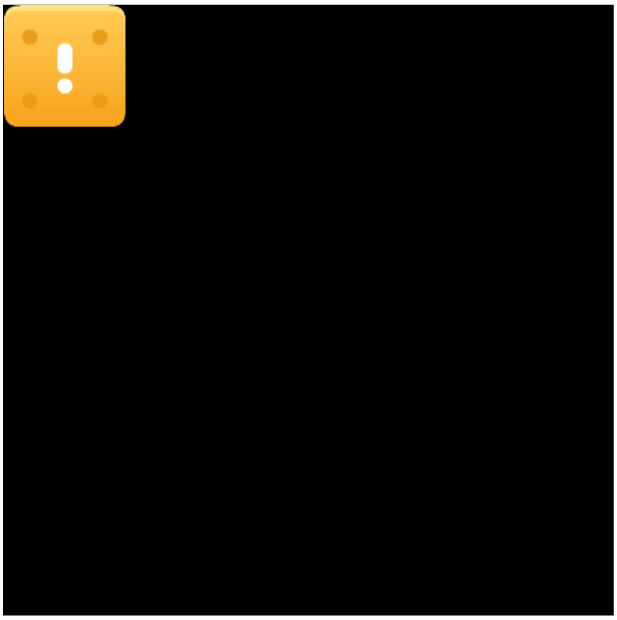


Figure 7: image

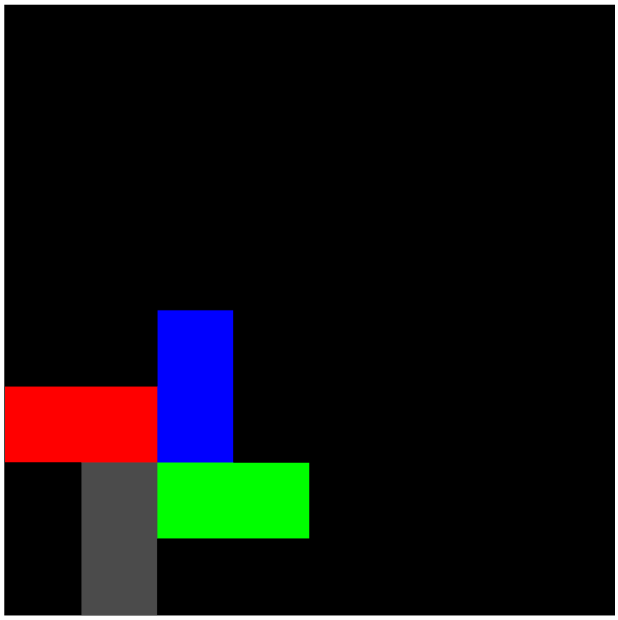


Figure 9: square-color